



# Resumen ejecutivo del SIIAA Laravel 13

## Arquitectura, autenticación, identidad institucional y autoría de datos

### 1. Descripción general

El **SIIAA Laravel 13** es la nueva versión del Sistema Integral de Información Académica y Administrativa del IRyA. Su objetivo es consolidar una arquitectura institucional moderna, modular y mantenible, capaz de integrar información proveniente de distintas fuentes internas y externas, particularmente:

- Personas SIIAA
- Estudiantes SIIAP
- Usuarios del sistema
- Roles y permisos
- Perfiles públicos
- Perfiles académicos
- Directorio institucional
- Módulos funcionales futuros

La versión actual se está construyendo sobre Laravel 13, Livewire, Blade, componentes UI propios, Tailwind, Alpine.js y Spatie Permission para control de roles y permisos.

Uno de los cambios arquitectónicos más importantes es la introducción de una **capa de identidad institucional**, basada en la tabla:

```
identity_links
```

Esta capa permite que distintos tipos de sujetos institucionales —personal IRyA, estudiantes SIIAP y futuras identidades— puedan participar en el sistema bajo una lógica común de propiedad, autoría, visibilidad y clasificación.

## 2. Objetivo arquitectónico principal

El objetivo central de la arquitectura actual es separar claramente cuatro conceptos que en versiones anteriores podían mezclarse:

| Concepto                | Representa                                                                                  |
|-------------------------|---------------------------------------------------------------------------------------------|
| Usuario                 | Cuenta que inicia sesión en el sistema                                                      |
| Identidad institucional | Persona, estudiante u otro sujeto institucional representado en <code>identity_links</code> |
| Propietario del dato    | Identidad institucional dueña del contenido o registro                                      |
| Autoría técnica         | Usuario que creó o modificó físicamente el registro                                         |

Esto permite que el sistema responda preguntas distintas sin ambigüedad:

```
¿Quién inició sesión?  
  users.id  
  
¿A qué identidad institucional corresponde?  
  identity_links.id  
  
¿A quién pertenece este registro?  
  owner_id = identity_links.id  
  
¿Quién creó o modificó el registro?  
  created_by / updated_by = users.id
```

## 3. Capas principales del sistema

La arquitectura funcional del SIIAA Laravel 13 puede entenderse en las siguientes capas:

```
Usuario autenticado  
↓
```

```
Autenticación local / LDAP / 2FA
↓
Roles y permisos
↓
Resolución de identidad institucional
↓
identity_links
↓
Módulos funcionales
↓
Propiedad institucional mediante owner_id
↓
Auditoría mediante created_by / updated_by
```

## 4. Capa de autenticación

La capa de autenticación determina **quién puede entrar al sistema**.

Actualmente contempla:

```
auth
verified
2fa.configured
identity.resolve
```

En rutas protegidas, el patrón general es:

```
Route::middleware([
    'auth',
    'verified',
    '2fa.configured',
    'identity.resolve',
])
```

### 4.1. **auth**

Valida que exista un usuario autenticado.

```
users.id
```

representa la cuenta de acceso al sistema.

## 4.2. **verified**

Garantiza que el usuario tenga correo verificado, cuando aplique.

## 4.3. **2fa.configured**

Asegura que el usuario cumpla con la política de doble factor de autenticación, si está habilitada.

## 4.4. **identity.resolve**

Es la capa clave del SIIAA. Su función es resolver si el usuario autenticado corresponde a una identidad institucional válida.

Esta identidad puede ser:

```
Personal IRyA registrado en Personas SIIAA  
Estudiante activo o vigente proveniente de SIIAP  
Otra identidad futura
```

# 5. Capa de autorización

La autorización se maneja mediante **Spatie Permission**.

El sistema distingue:

```
roles  
permissions
```

Ejemplos de permisos existentes o usados:

```
directorio.view  
directorio.update  
directorio.export  
  
personas.view
```

```
personas.create
personas.update
personas.delete
personas.manage_ingresos
personas.manage_public_profile
personas.manage_posdoc_becas

estudiantes.view
estudiantes.link_identity
estudiantes.manage_public_profile
```

## Regla general

La autenticación responde:

¿Quién eres?

La autorización responde:

¿Qué puedes hacer?

La identidad institucional responde:

¿A qué sujeto institucional representas?

---

## 6. Capa de identidad institucional

La tabla central de la capa de identidad es:

```
identity_links
```

Cada registro representa una identidad institucional operativa.

### Ejemplo: personal IRyA

```
identity_links.id = 10000025
identity_type = siiaa
identity_id = personas.id
email = correo@irya.unam.mx
active = true
```

## Ejemplo: estudiante SIAP

```
identity_links.id = 20000365
identity_type = siiap_student
identity_id = estudiantes.id
email = estudiante@correo.unam.mx
active = true
```

## Significado de campos principales

| Campo                      | Descripción                                                               |
|----------------------------|---------------------------------------------------------------------------|
| <code>id</code>            | Identificador único de la identidad institucional                         |
| <code>identity_type</code> | Tipo de identidad: <code>siiaa</code> , <code>siiap_student</code> , etc. |
| <code>identity_id</code>   | ID del registro en la fuente original                                     |
| <code>email</code>         | Correo usado para resolver o asociar identidad                            |
| <code>active</code>        | Indica si la identidad está activa para operación                         |
| <code>is_primary</code>    | Puede indicar identidad principal cuando exista más de una                |
| <code>matched_by</code>    | Forma de asociación: manual, email, proceso automático, etc.              |
| <code>matched_at</code>    | Fecha de asociación                                                       |
| <code>verified_at</code>   | Fecha de verificación                                                     |

## 7. Diferencia entre usuario e identidad

Un error común sería pensar que:

```
users.id = identidad institucional
```

Pero en SIIAA Laravel 13 no debe manejarse así.

**users.id**

Representa la cuenta técnica de acceso.

Ejemplo:

```
Usuario que inició sesión  
Usuario que creó un registro  
Usuario que modificó un registro
```

**identity\_links.id**

Representa al sujeto institucional.

Ejemplo:

```
Investigador IRyA  
Técnico académico  
Administrativo  
Posdoctorado  
Estudiante de maestría  
Estudiante de doctorado
```

## 8. Propiedad institucional de datos

Para módulos funcionales futuros, se acordó que el campo estándar para propiedad será:

```
owner_id
```

Y debe apuntar lógicamente a:

```
identity_links.id
```

## Regla general

```
owner_id = identity_links.id
```

No debe apuntar a:

```
users.id  
personas.id  
estudiantes.id
```

salvo que un módulo documente explícitamente una excepción.

## 9. Autoría técnica y auditoría

La autoría técnica se manejará con campos como:

```
created_by  
updated_by
```

Estos campos apuntan a:

```
users.id
```

Esto permite diferenciar:

```
Propietario institucional del dato  
owner_id = identity_links.id
```

```
Usuario que creó el dato  
created_by = users.id
```

```
Usuario que modificó el dato  
updated_by = users.id
```

## Ejemplo práctico



Un administrador captura un registro para un estudiante:

```
owner_id = identity_links.id del estudiante
created_by = users.id del administrador
updated_by = users.id del administrador
```

El dato pertenece institucionalmente al estudiante, aunque lo haya capturado un administrador.

## 10. Reglas de uso para módulos futuros

Todo módulo funcional que maneje información asociada a una persona, estudiante o sujeto institucional debe contemplar:

```
owner_id
created_by
updated_by
```

## Ejemplo de migración base

```
Schema::create('modulo_registros', function (Blueprint $table) {
    $table->id();

    /*
     * Propietario institucional.
     * Referencia lógica a identity_links.id.
     */
    $table->unsignedBigInteger('owner_id')->index();

    /*
     * Auditoría técnica.
     * Referencia lógica a users.id.
     */
    $table->unsignedBigInteger('created_by')->nullable()->index();
    $table->unsignedBigInteger('updated_by')->nullable()->index();

    $table->string('titulo');
    $table->text('descripcion')->nullable();

    $table->timestamps();
});
```

```
$table->softDeletes();  
});
```

## 11. Relación recomendada en modelos

Todo modelo con `owner_id` debería incluir:

```
public function owner()  
{  
    return $this->belongsTo(IdentityLink::class, 'owner_id', 'id');  
}
```

Y no:

```
belongsTo(User::class)  
belongsTo(Persona::class)  
belongsTo(Estudiente::class)
```

porque el propietario institucional puede venir de distintas fuentes.

## 12. Helper recomendado para identidad activa

Se recomienda normalizar la obtención de la identidad activa mediante helpers.

**activeIdentityLinkId()**

```
if (! function_exists('activeIdentityLinkId')) {  
    function activeIdentityLinkId(): ?int  
    {  
        return session('identity_link_id');  
    }  
}
```

## Uso

```
$ownerId = activeIdentityLinkId();
```

### currentIdentityLink()

```
if (! function_exists('currentIdentityLink')) {  
    function currentIdentityLink(): ?\App\Models\IdentityLink  
    {  
        $identityId = activeIdentityLinkId();  
  
        if (! $identityId) {  
            return null;  
        }  
  
        return \App\Models\IdentityLink::query()  
            ->where('active', true)  
            ->find($identityId);  
    }  
}
```

## Uso

```
$identity = currentIdentityLink();
```

### hasActiveIdentity()

```
if (! function_exists('hasActiveIdentity')) {  
    function hasActiveIdentity(): bool  
    {  
        return filled(activeIdentityLinkId());  
    }  
}
```

## Uso

```
if (! hasActiveIdentity()) {  
    abort(403, 'No tienes una identidad institucional asociada.');
```

## 13. Obtención del **owner\_id** en módulos futuros

Cuando un usuario crea un registro propio:

```
Registro::query()->create([  
    'owner_id' => activeIdentityLinkId(),  
    'titulo' => $this->titulo,  
    'descripcion' => $this->descripcion,  
]);
```

Cuando un módulo lista registros del usuario actual:

```
Registro::query()  
    ->where('owner_id', activeIdentityLinkId())  
    ->get();
```

## 14. Scope recomendado

Para evitar duplicar filtros:

```
public function scopeOwnedByCurrentIdentity($query)  
{  
    return $query->where('owner_id', activeIdentityLinkId());  
}
```

Uso:

```
Registro::ownedByCurrentIdentity()->get();
```

---

## 15. Vista personal vs vista administrativa

Los módulos deben distinguir entre:

```
Vista personal
Vista administrativa
```

### Vista personal

Filtra por identidad activa:

```
$query->where('owner_id', activeIdentityLinkId());
```

### Vista administrativa

Permite ver todos los registros si el usuario tiene permiso.

```
$query = Registro::query();

if (! auth()->user()->can('registros.view_all')) {
    $query->where('owner_id', activeIdentityLinkId());
}
```

---

## 16. Identidad e impersonation

La arquitectura contempla que en el futuro pueda existir **suplantación administrativa controlada**.

Reglas conceptuales acordadas:

```
Solo super-admin puede suplantar identidad.
Debe existir motivo obligatorio.
Debe ser temporal.
Debe mostrar indicador visual al super-admin.
```

```
Debe evitar acciones sensibles.  
Debe registrarse en logs.
```

Durante impersonation, es importante distinguir:

```
Usuario autenticado real  
Identidad institucional activa/suplantada
```

Ejemplo:

```
auth()->id()  
    usuario administrador real  
  
activeIdentityLinkId()  
    identidad institucional sobre la que está operando
```

## 17. Módulo Personas

El módulo Personas representa personal IRyA administrado desde SIIAA.

Rutas principales:

```
/personas  
/personas/crear  
/personas/{persona}  
/personas/{persona}/editar
```

La identidad asociada se representa como:

```
identity_links.identity_type = siiaa  
identity_links.identity_id = personas.id
```

## Información principal

```
Datos generales  
Ingresos institucionales
```

Perfil académico  
Perfil público  
Becas posdoctorales

## Decisión importante

Una persona puede tener historial de ingresos institucionales, pero debe existir un ingreso principal vigente.

La relación funcional para el ingreso actual es:

```
persona -> ingresoPrincipal
```

## 18. Módulo Estudiantes SIIAP

Los estudiantes provienen de SIIAP y se consultan en modo principalmente de solo lectura.

Ruta principal:

```
/estudiantes
```

La identidad asociada se representa como:

```
identity_links.identity_type = siiap_student  
identity_links.identity_id = estudiantes.id
```

## Criterio de estudiante activo

Un estudiante se considera activo o vigente si tiene inscripción dentro de los últimos tres semestres, considerando el semestre actual.

Ejemplo:

```
Semestre actual: 2026-2  
Últimos tres semestres:
```

2026-2  
2026-1  
2025-2

## Estados posibles

Inscrito actual  
En periodo de gracia  
No vigente

## 19. Módulo Directorio institucional

El Directorio se implementó como una vista única:

/directorio

Está basado en:

identity\_links  
perfiles\_publicos  
persona\_perfiles\_academicos

## Funciones principales

Listado editorial  
Filtro por búsqueda  
Filtro por tipo  
Modo edición inline  
Guardado por fila  
Guardado masivo  
Exportación CSV/JSON  
Feed público JSON/CSV

## Permisos



```
directorio.view  
directorio.update  
directorio.export
```

## 20. Perfil público

El perfil público se almacena en:

```
perfiles_publicos
```

Y se asocia mediante:

```
perfiles_publicos.identity_link_id = identity_links.id
```

## Campos principales

```
titulo_es  
titulo_en  
nombre_publico  
apellido_publico  
area_es  
area_en  
oficina  
extension_red_unam  
telefono_morelia  
telefono_cdmx  
email_publico  
homepage_url  
observaciones  
active  
visible  
sort_order  
directorio_tipo
```

## 21. Clasificación local del directorio

Para evitar cálculos pesados en cada request, se agregó:

```
perfiles_publicos.directorio_tipo
```

Este campo permite filtrar localmente sin recalcular datos desde SIIAP.

## Valores posibles

```
investigador  
tecnico_academico  
posdoctorado  
administrativo  
personal_confianza  
servicio_social  
estudiante_maestria  
estudiante_doctorado  
estudiante  
personal_iryá
```

## Regla importante

Los administrativos deben clasificarse explícitamente por clave de catálogo, no por fallback genérico.

```
administrativo ≠ personal_iryá  
administrativo ≠ personal_confianza  
administrativo ≠ servicio_social
```

## 22. Perfil académico

El perfil académico se almacena en:

```
persona_perfiles_academicos
```

Aunque conserva ese nombre, fue ajustado para asociarse mediante:

```
identity_link_id
```

Esto permite que tanto personas SIIAA como estudiantes SIIAP puedan tener perfil académico.

## Campos relevantes

```
orcid
scopus_id
sni_id
sni_vigencia
pride_id
pride_vigencia
ads_author_query
ads_profile_url
ads_library_url
research_area
academic_keywords
observaciones
```

## 23. Feed público del directorio

El feed público expone información visible para el sitio web institucional.

Rutas:

```
/api/public/directorio.json
/api/public/directorio.csv
```

Solo debe devolver registros que cumplan:

```
identity_links.active = true
perfiles_publicos.active = true
perfiles_publicos.visible = true
```

No debe exponer campos administrativos.

## 24. Mi perfil

La ruta:

```
/mi-perfil
```

usa el Livewire real:

```
app/Livewire/Personas/MiPerfil.php  
resources/views/livewire/personas/mi-perfil.blade.php
```

## Para personal IRyA

Muestra:

```
Datos generales  
Ingreso actual  
Perfil público completo  
Perfil académico sin datos ADS visibles
```

Permite editar únicamente:

```
homepage_url  
observaciones / semblanza
```

No permite modificar:

```
active  
visible  
sort_order  
directorio_tipo  
ADS  
datos generales  
ingreso
```

## Para estudiantes SIIAP

Muestra:

```
Datos generales
Perfil público completo
Inscripción actual
Comité tutor actual
```

No permite editar ningún campo.

## Comité tutor

La relación correcta es:

```
inscripcion_actual -> comite -> tutor
```

No debe usarse:

```
inscripciones.tutores
```

porque no existe en `EstudianteInscripcion`.

## 25. Flujo de interacción para módulos futuros

Un módulo futuro debe seguir este flujo:

1. Usuario inicia sesión.
2. Middleware valida autenticación, 2FA e identidad.
3. Se obtiene `identity_links.id` activo.
4. Al crear un registro:  
`owner_id = activeIdentityLinkId()`  
`created_by = auth()->id()`
5. Al modificar:  
`updated_by = auth()->id()`
6. Para vista personal:  
filtrar por `owner_id`.
7. Para vista administrativa:  
usar permisos.

8. Para clasificar datos:  
consultar `owner.identity_type`, `owner.identity_id` o relaciones derivadas.

## 26. Ejemplo práctico de creación

```
public function save(): void
{
    if (! hasActiveIdentity()) {
        abort(403, 'No tienes una identidad institucional activa.');
```

    }

```

    Registro::query()->create([
        'owner_id' => activeIdentityLinkId(),
        'titulo' => $this->titulo,
        'descripcion' => $this->descripcion,
        'created_by' => auth()->id(),
    ]);
}
```

## 27. Ejemplo práctico de consulta personal

```
public function render()
{
    return view('livewire.registros.index', [
        'registros' => Registro::query()
            ->where('owner_id', activeIdentityLinkId())
            ->latest()
            ->paginate(15),
    ]);
}
```

## 28. Ejemplo práctico de consulta administrativa

```
public function render()
{
    $query = Registro::query()
        ->with('owner');

    if (! auth()->user()->can('registros.view_all')) {
        $query->where('owner_id', activeIdentityLinkId());
    }

    return view('livewire.registros.index', [
        'registros' => $query->paginate(15),
    ]);
}
```

## 29. Clasificación de datos por identidad

Una vez que un registro tiene:

```
owner_id
```

se puede clasificar según la identidad propietaria.

Ejemplo:

```
$registro->owner->identity_type;
```

Valores:

```
siaa
siiap_student
```

Si se requiere clasificar más fino:

```
Investigador
Técnico académico
Administrativo
Posdoctorado
```

```
Estudiante de maestría  
Estudiante de doctorado
```

se puede consultar información derivada del propietario o usar campos locales calculados según el módulo.

---

## 30. Recomendación para módulos con clasificación frecuente

Si un módulo necesita filtrar frecuentemente por tipo de propietario, puede guardar un campo local derivado.

Ejemplo:

```
owner_type  
owner_category
```

Pero siempre manteniendo:

```
owner_id = identity_links.id
```

### Ejemplo

```
owner_id = 20000365  
owner_type = siiap_student  
owner_category = estudiante_doctorado
```

Esto evita recalcular información pesada en cada consulta.

---

## 31. Política de foreign keys

Para el SIIAA Laravel 13 se adoptó una política de FK selectivas:



## Usar FK reales en:

```
relaciones internas críticas  
relaciones estables  
tablas estrictamente controladas por SIIAA
```

## Evitar FK físicas en:

```
catálogos externos  
referencias flexibles  
created_by / updated_by  
owner_id basado en identidad flexible  
fuentes externas como SIIAP
```

En esos casos se recomienda:

```
unsignedBigInteger indexado  
validación desde Laravel/Eloquent  
relaciones Eloquent documentadas
```

## 32. Resumen ejecutivo final

El SIIAA Laravel 13 se organiza sobre una arquitectura modular donde la autenticación identifica al usuario, la autorización define lo que puede hacer, y la capa de identidad institucional determina a qué persona, estudiante o sujeto institucional representa.

La tabla `identity_links` es el eje común para integrar personas SIIAA y estudiantes SIIAP. En módulos futuros, la propiedad de los datos debe registrarse mediante `owner_id`, apuntando a `identity_links.id`. La autoría técnica debe mantenerse separada mediante `created_by` y `updated_by`, apuntando a `users.id`.

Esta separación permite clasificar, filtrar y auditar datos de forma robusta:

```
owner_id  
define a quién pertenece institucionalmente el dato
```

```
created_by / updated_by
    define qué usuario lo creó o modificó

identity_type / identity_id
    define de qué fuente institucional proviene

permisos
    definen quién puede consultar, editar o administrar
```

Esta arquitectura permite que los futuros módulos del SIIAA funcionen de forma homogénea para personal IRyA, estudiantes SIIAP y posibles identidades futuras, evitando acoplar la lógica funcional a tablas específicas como `personas`, `estudiantes` o `users`.

# Resumen ejecutivo del SIIAA Laravel 13

## Arquitectura, autenticación, identidad institucional y autoría de datos

### 1. Descripción general

El **SIIAA Laravel 13** es la nueva versión del Sistema Integral de Información Académica y Administrativa del IRyA. Su objetivo es consolidar una arquitectura institucional moderna, modular y mantenible, capaz de integrar información proveniente de distintas fuentes internas y externas, particularmente:

```
Personas SIIAA
Estudiantes SIIAP
Usuarios del sistema
Roles y permisos
Perfiles públicos
Perfiles académicos
Directorio institucional
Módulos funcionales futuros
```

La versión actual se está construyendo sobre Laravel 13, Livewire, Blade, componentes UI propios, Tailwind, Alpine.js y Spatie Permission para control de roles y permisos.

Uno de los cambios arquitectónicos más importantes es la introducción de una **capa de identidad institucional**, basada en la tabla:

```
identity_links
```

Esta capa permite que distintos tipos de sujetos institucionales —personal IRyA, estudiantes SIIAP y futuras identidades— puedan participar en el sistema bajo una lógica común de propiedad, autoría, visibilidad y clasificación.

## 2. Objetivo arquitectónico principal

El objetivo central de la arquitectura actual es separar claramente cuatro conceptos que en versiones anteriores podían mezclarse:

| Concepto                | Representa                                                                                  |
|-------------------------|---------------------------------------------------------------------------------------------|
| Usuario                 | Cuenta que inicia sesión en el sistema                                                      |
| Identidad institucional | Persona, estudiante u otro sujeto institucional representado en <code>identity_links</code> |
| Propietario del dato    | Identidad institucional dueña del contenido o registro                                      |
| Autoría técnica         | Usuario que creó o modificó físicamente el registro                                         |

Esto permite que el sistema responda preguntas distintas sin ambigüedad:

```
¿Quién inició sesión?  
users.id  
  
¿A qué identidad institucional corresponde?  
identity_links.id  
  
¿A quién pertenece este registro?  
owner_id = identity_links.id  
  
¿Quién creó o modificó el registro?  
created_by / updated_by = users.id
```

### 3. Capas principales del sistema

La arquitectura funcional del SIIAA Laravel 13 puede entenderse en las siguientes capas:

```
Usuario autenticado
↓
Autenticación local / LDAP / 2FA
↓
Roles y permisos
↓
Resolución de identidad institucional
↓
identity_links
↓
Módulos funcionales
↓
Propiedad institucional mediante owner_id
↓
Auditoría mediante created_by / updated_by
```

### 4. Capa de autenticación

La capa de autenticación determina **quién puede entrar al sistema**.

Actualmente contempla:

```
auth
verified
2fa.configured
identity.resolve
```

En rutas protegidas, el patrón general es:

```
Route::middleware([
    'auth',
    'verified',
    '2fa.configured',
    'identity.resolve',
])
```

## 4.1. **auth**

Valida que exista un usuario autenticado.

```
users.id
```

representa la cuenta de acceso al sistema.

## 4.2. **verified**

Garantiza que el usuario tenga correo verificado, cuando aplique.

## 4.3. **2fa.configured**

Asegura que el usuario cumpla con la política de doble factor de autenticación, si está habilitada.

## 4.4. **identity.resolve**

Es la capa clave del SIIAA. Su función es resolver si el usuario autenticado corresponde a una identidad institucional válida.

Esta identidad puede ser:

```
Personal IRyA registrado en Personas SIIAA  
Estudiante activo o vigente proveniente de SIIAP  
Otra identidad futura
```

# 5. Capa de autorización

La autorización se maneja mediante **Spatie Permission**.

El sistema distingue:

```
roles  
permissions
```

Ejemplos de permisos existentes o usados:

```
directorio.view
directorio.update
directorio.export

personas.view
personas.create
personas.update
personas.delete
personas.manage_ingresos
personas.manage_public_profile
personas.manage_posdoc_becas

estudiantes.view
estudiantes.link_identity
estudiantes.manage_public_profile
```

## Regla general

La autenticación responde:

¿Quién eres?

La autorización responde:

¿Qué puedes hacer?

La identidad institucional responde:

¿A qué sujeto institucional representas?

---

## 6. Capa de identidad institucional

La tabla central de la capa de identidad es:

```
identity_links
```

Cada registro representa una identidad institucional operativa.

## Ejemplo: personal IRyA

```
identity_links.id = 10000025
identity_type = siiaa
identity_id = personas.id
email = correo@irya.unam.mx
active = true
```

## Ejemplo: estudiante SIAP

```
identity_links.id = 20000365
identity_type = siiap_student
identity_id = estudiantes.id
email = estudiante@correo.unam.mx
active = true
```

## Significado de campos principales

| Campo                      | Descripción                                                               |
|----------------------------|---------------------------------------------------------------------------|
| <code>id</code>            | Identificador único de la identidad institucional                         |
| <code>identity_type</code> | Tipo de identidad: <code>siiaa</code> , <code>siiap_student</code> , etc. |
| <code>identity_id</code>   | ID del registro en la fuente original                                     |
| <code>email</code>         | Correo usado para resolver o asociar identidad                            |
| <code>active</code>        | Indica si la identidad está activa para operación                         |
| <code>is_primary</code>    | Puede indicar identidad principal cuando exista más de una                |
| <code>matched_by</code>    | Forma de asociación: manual, email, proceso automático, etc.              |
| <code>matched_at</code>    | Fecha de asociación                                                       |
| <code>verified_at</code>   | Fecha de verificación                                                     |

## 7. Diferencia entre usuario e identidad

Un error común sería pensar que:

```
users.id = identidad institucional
```

Pero en SIIAA Laravel 13 no debe manejarse así.

**users.id**

Representa la cuenta técnica de acceso.

Ejemplo:

```
Usuario que inició sesión  
Usuario que creó un registro  
Usuario que modificó un registro
```

**identity\_links.id**

Representa al sujeto institucional.

Ejemplo:

```
Investigador IRyA  
Técnico académico  
Administrativo  
Posdoctorado  
Estudiante de maestría  
Estudiante de doctorado
```

## 8. Propiedad institucional de datos

Para módulos funcionales futuros, se acordó que el campo estándar para propiedad será:



```
owner_id
```

Y debe apuntar lógicamente a:

```
identity_links.id
```

## Regla general

```
owner_id = identity_links.id
```

No debe apuntar a:

```
users.id  
personas.id  
estudiantes.id
```

salvo que un módulo documente explícitamente una excepción.

---

## 9. Autoría técnica y auditoría

La autoría técnica se manejará con campos como:

```
created_by  
updated_by
```

Estos campos apuntan a:

```
users.id
```

Esto permite diferenciar:

```
Propietario institucional del dato
    owner_id = identity_links.id
```

```
Usuario que creó el dato
    created_by = users.id
```

```
Usuario que modificó el dato
    updated_by = users.id
```

## Ejemplo práctico

Un administrador captura un registro para un estudiante:

```
owner_id = identity_links.id del estudiante
created_by = users.id del administrador
updated_by = users.id del administrador
```

El dato pertenece institucionalmente al estudiante, aunque lo haya capturado un administrador.

## 10. Reglas de uso para módulos futuros

Todo módulo funcional que maneje información asociada a una persona, estudiante o sujeto institucional debe contemplar:

```
owner_id
created_by
updated_by
```

## Ejemplo de migración base

```
Schema::create('modulo_registros', function (Blueprint $table) {
    $table->id();

    /*
     * Propietario institucional.
     * Referencia lógica a identity_links.id.
     */
});
```

```

$table->unsignedBigInteger('owner_id')->index();

/*
 * Auditoría técnica.
 * Referencia lógica a users.id.
 */
$table->unsignedBigInteger('created_by')->nullable()->index();
$table->unsignedBigInteger('updated_by')->nullable()->index();

$table->string('titulo');
$table->text('descripcion')->nullable();

$table->timestamps();
$table->softDeletes();
});

```

## 11. Relación recomendada en modelos

Todo modelo con `owner_id` debería incluir:

```

public function owner()
{
    return $this->belongsTo(IdentityLink::class, 'owner_id', 'id');
}

```

Y no:

```

belongsTo(User::class)
belongsTo(Persona::class)
belongsTo(Estudiante::class)

```

porque el propietario institucional puede venir de distintas fuentes.

## 12. Helper recomendado para identidad activa

Se recomienda normalizar la obtención de la identidad activa mediante helpers.

## activeIdentityLinkId()

```
if (! function_exists('activeIdentityLinkId')) {  
    function activeIdentityLinkId(): ?int  
    {  
        return session('identity_link_id');  
    }  
}
```

## Uso

```
$ownerId = activeIdentityLinkId();
```

## currentIdentityLink()

```
if (! function_exists('currentIdentityLink')) {  
    function currentIdentityLink(): ?\App\Models\IdentityLink  
    {  
        $identityId = activeIdentityLinkId();  
  
        if (! $identityId) {  
            return null;  
        }  
  
        return \App\Models\IdentityLink::query()  
            ->where('active', true)  
            ->find($identityId);  
    }  
}
```

## Uso

```
$identity = currentIdentityLink();
```

## hasActiveIdentity()

```
if (! function_exists('hasActiveIdentity')) {
    function hasActiveIdentity(): bool
    {
        return filled(activeIdentityLinkId());
    }
}
```

## Uso

```
if (! hasActiveIdentity()) {
    abort(403, 'No tienes una identidad institucional asociada.');
```

## 13. Obtención del **owner\_id** en módulos futuros

Cuando un usuario crea un registro propio:

```
Registro::query()->create([
    'owner_id' => activeIdentityLinkId(),
    'titulo' => $this->titulo,
    'descripcion' => $this->descripcion,
]);
```

Cuando un módulo lista registros del usuario actual:

```
Registro::query()
    ->where('owner_id', activeIdentityLinkId())
    ->get();
```

## 14. Scope recomendado

Para evitar duplicar filtros:

```
public function scopeOwnedByCurrentIdentity($query)
{
    return $query->where('owner_id', activeIdentityLinkId());
}
```

Uso:

```
Registro::ownedByCurrentIdentity()->get();
```

## 15. Vista personal vs vista administrativa

Los módulos deben distinguir entre:

```
Vista personal
Vista administrativa
```

### Vista personal

Filtra por identidad activa:

```
$query->where('owner_id', activeIdentityLinkId());
```

### Vista administrativa

Permite ver todos los registros si el usuario tiene permiso.

```
$query = Registro::query();

if (! auth()->user()->can('registros.view_all')) {
    $query->where('owner_id', activeIdentityLinkId());
}
```

## 16. Identidad e impersonation

La arquitectura contempla que en el futuro pueda existir **suplantación administrativa controlada**.

Reglas conceptuales acordadas:

```
Solo super-admin puede suplantar identidad.  
Debe existir motivo obligatorio.  
Debe ser temporal.  
Debe mostrar indicador visual al super-admin.  
Debe evitar acciones sensibles.  
Debe registrarse en logs.
```

Durante impersonation, es importante distinguir:

```
Usuario autenticado real  
Identidad institucional activa/suplantada
```

Ejemplo:

```
auth()->id()  
    usuario administrador real  
  
activeIdentityLinkId()  
    identidad institucional sobre la que está operando
```

## 17. Módulo Personas

El módulo Personas representa personal IRyA administrado desde SIIAA.

Rutas principales:

```
/personas  
/personas/crear  
/personas/{persona}  
/personas/{persona}/editar
```

La identidad asociada se representa como:

```
identity_links.identity_type = siiaa  
identity_links.identity_id = personas.id
```

## Información principal

```
Datos generales  
Ingresos institucionales  
Perfil académico  
Perfil público  
Becas posdoctorales
```

## Decisión importante

Una persona puede tener historial de ingresos institucionales, pero debe existir un ingreso principal vigente.

La relación funcional para el ingreso actual es:

```
persona -> ingresoPrincipal
```

# 18. Módulo Estudiantes SIIAP

Los estudiantes provienen de SIIAP y se consultan en modo principalmente de solo lectura.

Ruta principal:

```
/estudiantes
```

La identidad asociada se representa como:

```
identity_links.identity_type = siiap_student
```



```
identity_links.identity_id = estudiantes.id
```

## Criterio de estudiante activo

Un estudiante se considera activo o vigente si tiene inscripción dentro de los últimos tres semestres, considerando el semestre actual.

Ejemplo:

```
Semestre actual: 2026-2  
Últimos tres semestres:  
2026-2  
2026-1  
2025-2
```

## Estados posibles

```
Inscrito actual  
En periodo de gracia  
No vigente
```

# 19. Módulo Directorio institucional

El Directorio se implementó como una vista única:

```
/directorio
```

Está basado en:

```
identity_links  
perfiles_publicos  
persona_perfiles_academicos
```

## Funciones principales

```
Listado editorial
Filtro por búsqueda
Filtro por tipo
Modo edición inline
Guardado por fila
Guardado masivo
Exportación CSV/JSON
Feed público JSON/CSV
```

## Permisos

```
directorio.view
directorio.update
directorio.export
```

## 20. Perfil público

El perfil público se almacena en:

```
perfiles_publicos
```

Y se asocia mediante:

```
perfiles_publicos.identity_link_id = identity_links.id
```

## Campos principales

```
titulo_es
titulo_en
nombre_publico
apellido_publico
area_es
area_en
oficina
extension_red_unam
telefono_morelia
```

```
telefono_cdmx
email_publico
homepage_url
observaciones
active
visible
sort_order
directorio_tipo
```

## 21. Clasificación local del directorio

Para evitar cálculos pesados en cada request, se agregó:

```
perfiles_publicos.directorio_tipo
```

Este campo permite filtrar localmente sin recalcular datos desde SIAP.

### Valores posibles

```
investigador
tecnico_academico
posdoctorado
administrativo
personal_confianza
servicio_social
estudiante_maestria
estudiante_doctorado
estudiante
personal_iryia
```

### Regla importante

Los administrativos deben clasificarse explícitamente por clave de catálogo, no por fallback genérico.

```
administrativo ≠ personal_iryia
administrativo ≠ personal_confianza
administrativo ≠ servicio_social
```

---

## 22. Perfil académico

El perfil académico se almacena en:

```
persona_perfiles_academicos
```

Aunque conserva ese nombre, fue ajustado para asociarse mediante:

```
identity_link_id
```

Esto permite que tanto personas SIIAA como estudiantes SIIAP puedan tener perfil académico.

### Campos relevantes

```
orcid  
scopus_id  
sni_id  
sni_vigencia  
pride_id  
pride_vigencia  
ads_author_query  
ads_profile_url  
ads_library_url  
research_area  
academic_keywords  
observaciones
```

---

## 23. Feed público del directorio

El feed público expone información visible para el sitio web institucional.

Rutas:

```
/api/public/directorio.json
```

```
/api/public/directorio.csv
```

Solo debe devolver registros que cumplan:

```
identity_links.active = true  
perfiles_publicos.active = true  
perfiles_publicos.visible = true
```

No debe exponer campos administrativos.

---

## 24. Mi perfil

La ruta:

```
/mi-perfil
```

usa el Livewire real:

```
app/Livewire/Personas/MiPerfil.php  
resources/views/livewire/personas/mi-perfil.blade.php
```

## Para personal IRyA

Muestra:

```
Datos generales  
Ingreso actual  
Perfil público completo  
Perfil académico sin datos ADS visibles
```

Permite editar únicamente:

```
homepage_url  
observaciones / semblanza
```

No permite modificar:

```
active
visible
sort_order
directorio_tipo
ADS
datos generales
ingreso
```

## Para estudiantes SIIAP

Muestra:

```
Datos generales
Perfil público completo
Inscripción actual
Comité tutor actual
```

No permite editar ningún campo.

## Comité tutor

La relación correcta es:

```
inscripcion_actual -> comite -> tutor
```

No debe usarse:

```
inscripciones.tutores
```

porque no existe en `EstudianteInscripcion`.

## 25. Flujo de interacción para módulos futuros

Un módulo futuro debe seguir este flujo:

1. Usuario inicia sesión.
2. Middleware valida autenticación, 2FA e identidad.
3. Se obtiene `identity_links.id` activo.
4. Al crear un registro:  
`owner_id = activeIdentityLinkId()`  
`created_by = auth()->id()`
5. Al modificar:  
`updated_by = auth()->id()`
6. Para vista personal:  
filtrar por `owner_id`.
7. Para vista administrativa:  
usar permisos.
8. Para clasificar datos:  
consultar `owner.identity_type`, `owner.identity_id` o relaciones derivadas.

## 26. Ejemplo práctico de creación

```
public function save(): void
{
    if (! hasActiveIdentity()) {
        abort(403, 'No tienes una identidad institucional activa.');
```

  

```
    Registro::query()->create([
        'owner_id' => activeIdentityLinkId(),
        'titulo' => $this->titulo,
        'descripcion' => $this->descripcion,
        'created_by' => auth()->id(),
    ]);
}
```

## 27. Ejemplo práctico de consulta personal

```
public function render()
{
    return view('livewire.registros.index', [
        'registros' => Registro::query()
            ->where('owner_id', activeIdentityLinkId())
    ]);
}
```

```
        ->latest()  
        ->paginate(15),  
    ]);  
}
```

## 28. Ejemplo práctico de consulta administrativa

```
public function render()  
{  
    $query = Registro::query()  
        ->with('owner');  
  
    if (! auth()->user()->can('registros.view_all')) {  
        $query->where('owner_id', activeIdentityLinkId());  
    }  
  
    return view('livewire.registros.index', [  
        'registros' => $query->paginate(15),  
    ]);  
}
```

## 29. Clasificación de datos por identidad

Una vez que un registro tiene:

```
owner_id
```

se puede clasificar según la identidad propietaria.

Ejemplo:

```
$registro->owner->identity_type;
```

Valores:



```
siaa  
siiap_student
```

Si se requiere clasificar más fino:

```
Investigador  
Técnico académico  
Administrativo  
Posdoctorado  
Estudiante de maestría  
Estudiante de doctorado
```

se puede consultar información derivada del propietario o usar campos locales calculados según el módulo.

## 30. Recomendación para módulos con clasificación frecuente

Si un módulo necesita filtrar frecuentemente por tipo de propietario, puede guardar un campo local derivado.

Ejemplo:

```
owner_type  
owner_category
```

Pero siempre manteniendo:

```
owner_id = identity_links.id
```

### Ejemplo

```
owner_id = 20000365  
owner_type = siiap_student
```

```
owner_category = estudiante_doctorado
```

Esto evita recalcular información pesada en cada consulta.

---

## 31. Política de foreign keys

Para el SIIAA Laravel 13 se adoptó una política de FK selectivas:

### Usar FK reales en:

```
relaciones internas críticas  
relaciones estables  
tablas estrictamente controladas por SIIAA
```

### Evitar FK físicas en:

```
catálogos externos  
referencias flexibles  
created_by / updated_by  
owner_id basado en identidad flexible  
fuentes externas como SIIAP
```

En esos casos se recomienda:

```
unsignedBigInteger indexado  
validación desde Laravel/Eloquent  
relaciones Eloquent documentadas
```

---

## 32. Resumen ejecutivo final

El SIIAA Laravel 13 se organiza sobre una arquitectura modular donde la autenticación identifica al usuario, la autorización define lo que puede hacer, y la capa de identidad institucional determina a qué persona, estudiante o sujeto institucional representa.

La tabla `identity_links` es el eje común para integrar personas SIIAA y estudiantes SIAP. En módulos futuros, la propiedad de los datos debe registrarse mediante `owner_id`, apuntando a `identity_links.id`. La autoría técnica debe mantenerse separada mediante `created_by` y `updated_by`, apuntando a `users.id`.

Esta separación permite clasificar, filtrar y auditar datos de forma robusta:

```
owner_id
    define a quién pertenece institucionalmente el dato

created_by / updated_by
    define qué usuario lo creó o modificó

identity_type / identity_id
    define de qué fuente institucional proviene

permisos
    definen quién puede consultar, editar o administrar
```

Esta arquitectura permite que los futuros módulos del SIIAA funcionen de forma homogénea para personal IRyA, estudiantes SIAP y posibles identidades futuras, evitando acoplar la lógica funcional a tablas específicas como `personas`, `estudiantes` o `users`.